Using Random Testing Tools

Members:
Dhruv Patel
Vraj Patel

1. Introduction **3**
1.1 Selected Projects **3**
2. Task 1: Fuzzing with LibFuzzer **4**
2.1 Project Selection **4**
2.2 Function Selection **5**
2.3 Fuzzing Harness and Setup **5**
2.4 Fuzzing Process **6**
2.5 Coverage Analysis **14**
2.6 Challenges **15**
2.7 Reflection on Fuzzing Process and Tools **16**
3. Task 2: Fuzzing with AFLplusplus **17**
3.1 Project Selection **17**
3.2 Function Selection **17**
3.3 Fuzzing Harness and Setup **18**
3.4 Fuzzing Process **18**
3.5 Results and Analysis **21**
3.6 Challenges and Reflection **23**
3.7 Reflection on Fuzzing Process and Tools **24**
4. Comparison Between LibFuzzer and AFLplusplus **24**
5. Overall Analysis **27**
6. Individual Analysis **28**
7. Conclusion **29**

1. Introduction

Fuzz testing is a critical technique in software development and security analysis, designed to uncover unexpected or harmful behaviors in software systems. This assignment focuses on applying fuzz testing to open-source projects to identify potential security vulnerabilities and robustness issues that might escape traditional testing methods.

Our objectives for this fuzzing exercise are:

- To evaluate the effectiveness of fuzz testing in identifying vulnerabilities in real-world open-source projects.
- To compare the performance and capabilities of two popular fuzz testing tools: LibFuzzer and AFLplusplus.
- To gain practical experience in setting up and running fuzz tests on substantial codebases
- To analyze the results of fuzz testing and draw insights for improving software security and robustness.

1.1 Selected Projects

For this exercise, we have chosen two different open-source projects:

1. http-parser (C): A parser for HTTP messages written in C, which we analyzed using LibFuzzer.
    - OpenHub URL: https://openhub.net/p/joyent-http-parser
    - GitHub URL: https://github.com/nodejs/http-parser
    - Size: 6.74K lines of code
    - Target Function: http_parser_execute
    - Tool used: Libfuzzer (Task 1)
2. JSON-C (C): A JSON implementation in C, which we analyzed using AFLplusplus.
    - OpenHub URL: https://openhub.net/p/json_c
    - GitHub URL: https://github.com/json-c/json-c
    - Size: 10.6k lines of code
    - Target Function: json_tokener_parse
    - Tool used: AFLplusplus (Task 2)

We selected these projects based on requirements per the exercise question which includes more than 4000 lines of code and an existing testing suite making them ideal for fuzzing.

We tested a few different other projects before finalizing these ones. The reason for testing other projects before picking one was because of the challenges associated with setting up fuzz-testing environments. Many open-source projects we initially considered presented various obstacles such as outdated dependencies or build systems, and lack of active maintenance making it

difficult to address any compilation issues. Additionally, the lack of knowledge we have on complex architectures would require significant modifications. Critical parsing functions were our primary aim to ensure we get efficient results.

For most projects we tested, we found out that they already included fuzzers, which made it difficult for us to identify untested code paths since these existing fuzzers had already covered a significant portion of the codebase. Hence, we decided to use existing fuzzers (particularly for Task 1, and modify them to provide extended coverage. We used our fuzzing harness to increase the diversity of inputs and explored edge cases that the original fuzzers might not have covered. By running the fuzzer for an extended duration, we were able to generate a more comprehensive set of test cases and exercise code paths that might be rare.

Since this exercise aims to learn and gain hands-on experience with fuzz testing and improving fuzzing strategies, we focused on understanding how to set up a comprehensive fuzzing harness, how operators work, and how to maximize code coverage. Even though finding new vulnerabilities was challenging due to the thoroughness of existing fuzzers.

2. Task 1: Fuzzing with LibFuzzer

2.1 Project Selection

HTTP Parser is a high-performance C-based library designed for parsing HTTP requests and responses. It is particularly useful for applications where efficiency is crucial, such as web servers and network-based programs. The parser is compact, requiring minimal memory (about 40 bytes per message stream), and it operates without making any system calls or memory allocations. This makes it ideal for scenarios that demand low latency and high throughput.

HTTP Parser operates by initializing a http_parser object per TCP connection and configuring callbacks to handle various parts of the HTTP message. It processes data incrementally, making it well-suited for asynchronous or interrupted streams. The parser supports advanced features like protocol upgrades and can be used in multithreaded environments.

(Reference: https://llhttp.org/)

Below is the list of criteria we identified from the project:

- **Size of the code base**: 6.74K
- **Operating System**: Ubuntu 22.04.4 LTS (set as a virtual machine through WSL)
- **Language**: C

- **Build tool**: make
- **Build time to compile and link an executable from source code**: 16.998s
- **Test Suit Infrastructure**:
    - **Number of test files**: 1
    - **Number of tests**: 114 functions (grep -c "test_" test.c)
    - **Lines of test code**: 4729 (wc -l test.c)
    - **Execution time for the entire suite**: 7.902s (time ./test_g)

2.2 Function Selection

The http_parser_execute function is a critical component of the http-parser library, responsible for parsing HTTP messages. It parses HTTP messages (both requests and responses) incrementally, processing the input data byte by byte.

Based on the function, it takes the following parameters:

- http_parser parser: A pointer to the parser structure
- const http_parser_settings settings: Callback functions for various parsing events
- const char data: The input data to be parsed
- size_t len: The length of the input data

Based on our understanding, the function is designed to be efficient, using lookup tables and bit operations for fast character classification. It includes checks to prevent buffer overflows and enforce limits on header sizes.

As a parser for network protocols, it's a common target for potential security vulnerabilities. Bugs in this function could lead to security issues like buffer overflows making it an ideal choice for us to test.

This was important for us to understand to configure the harness script for fuzzing accordingly.

2.3 Fuzzing Harness and Setup

We modified the existing fuzzing harness to target the http_parser_execute function. The harness is designed to simulate the parsing of HTTP messages with various inputs. Below is a snippet of our fuzzing harness:

```c
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
  static const http_parser_settings settings_null = {
    .on_message_begin = 0,
    .on_header_field = 0,
    .on_header_value = 0,
    .on_url = 0,
    .on_status = 0,
    .on_body = 0,
    .on_headers_complete = 0,
    .on_message_complete = 0
  };

  http_parser parser;
  http_parser_init(&parser, HTTP_BOTH);
  http_parser_execute(&parser, &settings_null, (const char*)data, size);
  return 0;
}
```

The harness is designed to:

- Initialize an HTTP parser allowing it to parse both requests and responses. This is to maximize the coverage of the parsing logic.
- Set up null callbacks for all parsing events. This helps us to fuzz the core parsing logic.
- Execute the parser with fuzz-generated input data. The data and size parameters are provided by LibFuzzers and represent the fuzzed input.

We decided to keep the harness simple, yet effective to allow LibFuzzer to thoroughly explore the function's behavior across a wide range of inputs, potentially finding bugs, memory issues, or any other vulnerabilities in the HTTP parsing logic.

(For a detailed view, please look at the files provided in the artifacts.)

2.4 Fuzzing Process

For our fuzzing, we outlined the steps below:

2.4.1 Inputs and Seed Files

To improve the fuzzing process, we filled the corpus folder with real-world HTTP messages to act as initial seed files to guide LibFuzzer in exploring different paths. We aimed to provide it with diverse and complex examples, both typical and with bugs to maximize code coverage. We downloaded these files from two GitHub repositories that provide a wide range of examples related to HTTP-related content:

1) FuzzDB Attack Patterns:
    - GitHub link: https://github.com/fuzzdb-project/fuzzdb/tree/master/attack
    - This repository contains known attack pattern sequences for intelligent brute force testing. It includes patterns from various sources such as:
        - scanner patterns from HTTP logs
        - books, articles, blog posts
        - other open-source fuzzers
        - error messages
        - Application documentations

    (Reference: here)

2) HTTPcorpus:
    - GitHub link: https://github.com/personnumber3377/HTTPcorpus
    - This corpus is specifically designed for HTTP fuzzing and includes:
        - Random content and filenames generated using custom tools
        - Grammar-based mutations

We populated our corpus folder with a total of **123** files. These files are included in the submission folder.

Here are some examples of the files we used to help us understand why this is important:

File Type/Name	Description	File Content
http-protocol-methods.txt	List of HTTP methods, including common and complex ones	OPTIONS GET HEAD POST PUT DELETE TRACE ...
crlf-injection.txt	CRLF injection patterns for testing response splitting vulnerabilities	%0d%0a %0d%0a%0d%0a r%0d%0aContentLength:%200%0d%0a%0d%0aHTTP/1.1%20200%20OK%0d%0aContentType:%20text/html%0d%0aContentLength:%2019%0d%0a%0d%0a<html>Injected%02Content</html> %0d%0d%0a%0a 0x0D0x0A

known-uri-types.txt	Known URL schemes to test URL parsing	aaa: aaas: about: acap: adiumxtra: ...
GET Request	Standard HTTP GET request	GET / HTTP/1.1 Host: 127.0.0.1:8080 Connection: keep-alive sec-ch-ua: "Not A(Brand";v="24", "Chromium";v="110" ...
PUT Request	Malformed PUT request with corrupted data	PUT /j.png HTTP/1.1 Host: l User-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64; hRKweEdWSYQEUDkzwqq
Complex URL Example	A URL example with complex parameters	UNBIND ftp://9:o@127.0.0.1:8080/W/o/#B HTTP/0.9 L

These are only some of the important examples that we chose to show in the report to help us understand what kind of files we have used. To look at the additional files, please refer to the corpus folder.

(Please note, for most of these files we did not know the exact usage. Hence, we have described them based on the file names and references that have been provided).

(Reference: here)

2.4.2 Execution (Fuzzing Configuration)

We set up a comprehensive configuration based on the tutorial to maximize the effectiveness of LibFuzzer.

1. **Building the Fuzzer**:
   - We compiled our fuzzed using clang with specific flags:

```
clang -g -fsanitize=fuzzer,address,signed-integer-overflow -fprofile-instr-generate -fcoverage-mapping -I.
fuzzers/fuzz_parser.c http_parser.c -o fuzz_parser
```

   - -fsanitize=fuzzer,address,signed-integer-overflow: Enables LibFuzzer, AddressSanitizer, and signed integer overflow checking

2. **Executing the Fuzzer**:

- We ran the fuzzer for an extended period to thoroughly explore (48 hours):

```
./fuzz_parser -print_final_stats=1 -artifact_prefix=./crashes/ -max_total_time=172800 corpus/
```

- Additionally, we added the LLVM_PROFILE_FILE="coverage.profraw" to store the collected data. This was important to us to generate a coverage report.

```
vrajp@Vraj:~/report/http-parser$ LLVM_PROFILE_FILE="coverage.profraw" ./fuzz_parser -print_final_stats=1 -artifact_prefix=./crashes/ -max_total_time=172800 corpus/
INFO: Seed: 208709943
INFO: Loaded 1 modules   (961 inline 8-bit counters): 961 [0x5d062c, 0x5d09ed),
INFO: Loaded 1 PC tables (961 PCs): 961 [0x58c6c0,0x5902d0),
INFO:      870 files found in corpus/
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 20804 bytes
INFO: seed corpus: files: 870 min: 1b max: 20804b total: 145641b rss: 27Mb
#871    INITED cov: 358 ft: 1010 corp: 348/40Kb exec/s: 0 rss: 45Mb
#872    NEW    cov: 358 ft: 1011 corp: 349/41Kb lim: 615 exec/s: 0 rss: 45Mb L: 187/615 MS: 1 CrossOver-
#898    REDUCE cov: 358 ft: 1011 corp: 349/41Kb lim: 615 exec/s: 0 rss: 45Mb L: 121/615 MS: 1 EraseBytes-
#909    REDUCE cov: 358 ft: 1011 corp: 349/41Kb lim: 615 exec/s: 0 rss: 45Mb L: 50/615 MS: 1 EraseBytes-
#1105   REDUCE cov: 358 ft: 1011 corp: 349/41Kb lim: 615 exec/s: 0 rss: 45Mb L: 117/615 MS: 1 EraseBytes-
#1251   REDUCE cov: 358 ft: 1011 corp: 349/40Kb lim: 615 exec/s: 0 rss: 45Mb L: 255/615 MS: 1 EraseBytes-
#1382   NEW    cov: 358 ft: 1014 corp: 350/41Kb lim: 615 exec/s: 0 rss: 45Mb L: 277/615 MS: 1 CopyPart-
#1493   REDUCE cov: 358 ft: 1014 corp: 350/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 45/615 MS: 1 EraseBytes-
#1537   REDUCE cov: 358 ft: 1014 corp: 350/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 407/615 MS: 4 InsertByte-InsertByte-ChangeByte-EraseBytes-
#1583   REDUCE cov: 358 ft: 1014 corp: 350/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 83/615 MS: 1 EraseBytes-
#1625   NEW    cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 121/615 MS: 2 CopyPart-ChangeByte-
#1681   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 74/615 MS: 1 EraseBytes-
#1768   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 34/615 MS: 2 ChangeByte-EraseBytes-
#1790   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 152/615 MS: 2 CopyPart-EraseBytes-
#1792   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 147/615 MS: 2 ChangeBit-EraseBytes-
#1945   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 158/615 MS: 3 ChangeASCIIInt-InsertByte-EraseBytes-
#2047   REDUCE cov: 358 ft: 1015 corp: 351/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 27/615 MS: 2 ChangeByte-EraseBytes-
#2145   NEW    cov: 358 ft: 1034 corp: 352/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 515/615 MS: 3 ChangeBit-ChangeBinInt-CopyPart-
#2153   REDUCE cov: 358 ft: 1034 corp: 352/41Kb lim: 615 exec/s: 0 rss: 46Mb L: 99/615 MS: 3 CopyPart-InsertByte-EraseBytes-
```

3. **Post-Fuzzing Analysis**:
   - After the fuzzing session was over, we performed some additional steps to analyze the results:
   - Merging the coverage data:

```
llvm-profdata merge --sparse coverage.profraw --output coverage.profdata
```

   - Generating an HTML coverage report:

```
llvm-cov show ./fuzz_parser -instr-profile=coverage.profdata -format=html -output-dir=coverage-html
```

(Reference we used to learn this: here)

2.4.3 Libfuzzer's Operation

During the fuzzing, LibFuzzer used several operators (mutation) to generate a diverse set of inputs for the testing the http_parser_execute function. These operators helped to discover potential vulnerabilities or unexpected behaviours.

Some of the operators we noticed are:

- CopyPart
- EraseBytes
- ChangeASCIIInt
- EraseByte
- InsertByte

- ChangeByte
- InsertRepeatedBytes
- ChangeBit
- ShuffleBytes

These are just some of the operators. LibFuzzer used several others during the 48 hour run. For simplicity, we only mentioned a few to show how LibFuzzer is creating the wide range of inputs by exploring different input lengths, testing boundary conditions and edge cases, uncovering potential integer-related cases or identifying issues with specific byte patterns. In some changes, LibFuzzer combined more than one operator for a single input file.

Initially, we ran the LibFuzzer for 4 hours to ensure it is generating outputs and working as expected. We noticed a recommended dictionary that was shown in the terminal:

```
###### Recommended dictionary. ######
"\xff\xff\xff\xff\xff\xff\x03\x0c" # Uses: 84761
"\x02\xe7" # Uses: 77118
"\x00\x00\x00\x06" # Uses: 76328
"\x00\x00\x00\x07" # Uses: 76969
"5\x00\x00\x00" # Uses: 76654
"\x00\x00\x00\x00\x00\x00\x00O" # Uses: 74807
"\x01\x01@\x0a" # Uses: 71020
"\xff\xff\xff'" # Uses: 70582
"\x09@\x01\x00" # Uses: 68452
"\x01\x00\x00\x00" # Uses: 67173
"&\x00\x00\x00" # Uses: 62037
":\x00\x00\x00" # Uses: 33613
"\xff:" # Uses: 23464
"\x00\x01@\x0a" # Uses: 22500
"\xf0\x00\x00\x00\x00\x00\x00\x00" # Uses: 22452
"\x1f\x00\x00\x00" # Uses: 20563
"\xe2'\x00\x00\x00\x00\x00\x00" # Uses: 19055
"\xff\xff\xff\x0a" # Uses: 17966
"\x00\x00\x00\x00" # Uses: 15946
"\x15\x00\x00\x00" # Uses: 15090
";\x00" # Uses: 11752
"\x81\x00\x00\x00" # Uses: 7040
###### End of recommended dictionary. ######
```

Upon some research, libFuzzer can use these dictionaries to improve its performance in the following runs. We should collect these recommended dictionaries from each run and incrementally build a larger dictionary that is fed as input for the next fuzzing round. The image contains a small portion of the dictionary, the actual recommended dictionary was more than 100 lines.

Additionally, the **Uses** indicates how many times that particular byte sequence was used during the fuzzing session. A higher number suggests that the sequence is particularly effective or relevant for exploring new paths in the code or hitting interesting cases which is why we decided to include it in the next longer run.

(Reference: here)

Hence, this was our updated command to run the fuzzer:

```
LLVM_PROFILE_FILE="coverage.profraw" ./fuzz_parser -print_final_stats=1 -artifact_prefix=./crashes/ -max_total_time=172800 -
dict=suggest_dic.dict corpus/
```

2.4.4 Analysis of Fuzzing Results

After running the fuzzer for 48 hours, **libFuzzer was not able to find any crashes or slow units.** Most open-source projects already have a robust codebase with existing fuzzers set. Hence, fuzzing coverage may be limited. Http-parser is already a well-developed project with complex input requirements that have been satisfied through testing methods.

However, for this exercise, we will still analyse other aspects of the project based on the results such as:

- Code Coverage
- Fuzzer Statistics
- Corpus Analysis
- Fuzzer logs
- Resource usage
- Malformed Inputs
- Corpus Growth
- Fuzzing Efficiency

2.4.5 Metrics and Statistics

After running for 48 hours, LibFuzzer provided valuable statistics that helped us understand the effectiveness and performance of the function:

```
Done 22022866026 runs in 172801 second(s)
stat::number_of_executed_units: 22022866026
stat::average_exec_per_sec:     127446
stat::new_units_added:          38293
stat::slowest_unit_time_sec:    0
stat::peak_rss_mb:              859
LLVM_PROFILE_FILE="coverage.profraw" ./fuzz_parser -print_final_stats=1     168611.67s user 3613.81s system 99% cpu 48:00:01.60 total
```

1) Total Executions: 22022866026
    - This high number of executions indicates a thorough through use of the input files
2) Average Executions per second: 127446
    - This execution rate suggests the efficient performance of our fuzzer and target function.
3) New Units Added: 38293
    - These are inputs that triggered new code paths, expanding our coverage.

4) Slowest Unit Time: 0
    - This shows us that not a single input caused a significant slowdown in execution
5) Peak RSS: 859 MB
    - This shows us the maximum memory usage.

These metrics demonstrate that our fuzzing campaign was both extensive and efficient, with a high execution rate and a significant number of new code paths discovered.

2.4.6 Results and Observations

After an initial run of 4 hours, the fuzzer generated **4926** files under corpus. After 48 hours, the fuzzer generated **41061** number of files under corpus.

We chose 10 files used by LibFuzzer that led to interesting behaviours and increased code coverage. Analyzing these files will help us understand the edge cases and how Libfuzzer is mutating different files to use them as inputs.

LibFuzzer generated a wide variety of malformed and unexpected HTTP requests and messages, which provided valuable insights into the robustness of the http-parser project.

	Observation	File Content

1	This input tests the parser's ability to handle multiple, malformed UPGRADE headers. It checks for potential vulnerabilities in header parsing and memory management when dealing with repeated, non-standard headers.	`PUT /`
		`UPGRADEY:DEY:AE/`
		`UPGRADEY:T /`
		`UPGRADEY:GP:/`
		`UPGRADEY:P`
		`...`

2	This input examines how the parser handles multiple CONNECTION headers with the UPGRADE value. It also checks for buffer overflow.	`PUT /e045d`
		`CONNECTion:UPGRADE`
		`CONNECTion:UPGRADE`
		`CONNECTion:UPGRADE`
		`...`

3	This complex input combines different HTTP versions, UPGRADE headers, and CONNECTION headers. It tests the parser's version handling against malformed status lines.	`HTTP/6.1 184 1/`
		`UPGRADE:T /ELE`
		`CONNECTion:UPGRADE`
		`...`

4	These extremely short HTTP responses test the parser's ability to handle minimal valid inputs and its robustness against incomplete responses.	HTTP/4.2 185 HTTP/4.2 4
5	This input explores how the parser handles a series of non-standard headers with similar names.	PUT / ConGt: Con3: ConC: Con0: Conon: ...
6	These requests test the parser's URL handling, particularly with fragment identifiers (#) and different HTTP versions. It checks for proper URL parsing and version compatibility.	PUT /#& HTTP/2.4 PUT /#H HTTP/1.4
7	This short, malformed GET request tests the parser's error handling for incomplete headers and unexpected characters at the beginning of the request.	d6 GET /p.1 ecCon:
8	It tests the parser's ability to handle incomplete data and maintain a consistent state with headers cut off.	60 GET / HTTP/2.1 Host: 127.0.0.1:8080 Connection: keep-alive ...
9	This input tests the parser's handling of multiple Host headers, which is invalid according to the HTTP specification. It checks for proper error handling and potential security issues.	9a PUT / Host: i Ter: Tgen: ...
10	This input tests the parser's robustness against malformed POST requests with unusual characters in the URL and repeated empty Host headers.	62 POST /kc$<=/PH3T.1 Host: Host: nne

These are only some of the files. For some other files, you may refer to our corpus folder. Additionally, the file content consists of several lines, so we decided to show the first few for each to get an understanding.

Our Key Observations:

- LibFuzzer demonstrated its ability to generate a wide range of inputs, from minimal edge cases to complex, structured data. This is crucial for thorough testing of the HTTP parser.
- Several inputs tested the parser's ability to handle different HTTP versions, including invalid ones. This helps ensure robust version parsing.
- Inputs like 1, 2, and 3 tested the parser's handling of repeated headers and malformed header values, which is important for security and compliance with the HTTP specification.
- Inputs 4 and 8 focused on testing URL parsing with various edge cases.
- Short inputs tested the parser's behaviour with minimal or incomplete data, which is crucial for malformed requests.
- Many of these inputs are malformed in various ways, testing the parser's error handling against invalid inputs.
- These inputs help verify that the parser correctly implements the HTTP protocol specifications, especially in handling various headers and request formats.

2.5 Coverage Analysis

From the HTML report generated, here are the results:

Filename	Function Coverage	Line Coverage	Region Coverage	Branch Coverage
fuzzers/fuzz_parser.c	100.00% (1/1)	100.00% (18/18)	100.00% (1/1)	- (0/0)
http_parser.c	27.78% (5/18)	66.60% (1045/1569)	49.97% (1663/3328)	55.26% (967/1750)
Totals	**31.58% (6/19)**	**66.98% (1063/1587)**	**49.98% (1664/3329)**	**55.26% (967/1750)**

Here are our observations from the report:

- The fuzzing harness (fuzz_parser.c) achieved 100% coverage across all measured metrics, indicating that our fuzzer is fully working.
- From the http_parser.c:
    - Function coverage is relatively low at 27.78%, suggesting that many functions in the parser are not being reached by our current fuzzing setup. However, since we are testing only one function from the file, it is a relatively understandable number.
    - Line coverage is moderate at 66.60%, indicating that a significant portion of the code is being exercised.

    - Region and branch coverage is around 49-55%, showing that about half of the code regions and branches are being explored.
- Overall, the fuzzing has achieved moderate coverage, with room for improvement, particularly in function coverage.

Additionally, it suggests that the fuzzer is effectively exploring many paths with room for improvement. Since we only ran our fuzzer for 48 hours, this should be significant.

2.6 Challenges

Initial Challenges:

1. Setting up the fuzzing environment:
    - Configuring LibFuzzer with the correct compiler flags and sanitizers required careful attention. For example, Address Sanitizer and Memory Sanitizer could not be combined and would need to run separately. Hence, choosing the right flags was a challenge.
    - Ensuring all dependencies were properly compiled for http-parser took a significant amount of time.
2. Creating and understanding fuzzing harness:
    - Designing a harness that adequately exercised the http_parser_execute function while avoiding false positives was challenging.
    - Balancing between thorough testing and avoiding unnecessary computations in the harness required some detailed understanding on how the function works.

Performance:

1. Fuzzing speed:
    - Balancing between fuzzing speed and thorough testing of complex inputs was an ongoing challenge.
2. Memory:
    - Managing memory usage, especially with Address Sanitizer enabled, required careful monitoring to prevent out-of-memory errors.
3. Disk:
    - Handling a large number of generated test cases without overwhelming the file system required implementing efficient corpus management strategies such as ensuring all the files are saved in the corpus folder.

Fuzzing Strategies:

1. Coverage:

- After initial runs, reaching new code paths became increasingly difficult, hence we had to check the harness and ensure it does the right thing.

2.7 Reflection on Fuzzing Process and Tools

Strengths of LibFuzzer:

1. Integration with several different sanitizers:
    - The seamless integration with AddressSanitizer and UndefinedBehaviorSanitizer greatly enhanced our ability to run fuzzer without simultaneous runs.
2. Coverage-guided technique:
    - LibFuzzer's ability to prioritize inputs that reach new code paths significantly improved the efficiency of our fuzzing.
3. Speed:
    - LibFuzzer allowed for very high execution speeds, enabling us to test a large number of inputs quickly. More than 40000 files were created by LibFuzzer during it's 48 hour run.

Weaknesses and Limitations:

1. Limited to in-process fuzzing:
    - A significant challenge is that Libfuzzer is unable to fuzz entire programs or systems, limiting our testing to specific functions.
2. Dependency on initial inputs:
    - The effectiveness of the fuzzing was heavily influenced by the quality of our initial input files that we saved under corpus.
3. Getting stuck on certain paths:
    - Sometimes the fuzzer would focus too much on a particular code path.
4. Complexity in configuring:
    - Finding the right configuration for LibFuzzer required significant experimentation and knowledge. Before finalizing one project and one function, we had to test various functions to understand how fuzzing works.

3. Task 2: Fuzzing with AFLplusplus

3.1 Project Selection

JSON-C is a library written in C that enables the construction, parsing, and manipulation of JSON formatted data in C. The library provides a reference-counted object model to simplify JSON data handling in C applications.

Below is the list of criteria we identified from the project:

- **Size of the code base**: 10,632 lines of code
- **Operating System**: Ubuntu 22.04.4 LTS (set as a virtual machine through WSL)
- **Language**: C
- **Build tool**: CMake
- **Build time to compile and link an executable from source code**: 0.934 seconds
- **Test Suit Infrastructure**:
    - **Number of test files**: 25
    - **Number of tests**: 46
    - **Lines of test code**: 7990
    - **Execution time for the entire suite**: 3.44 seconds

3.2 Function Selection

The json_tokener_parse function is a key component of the JSON-C library, responsible for parsing JSON strings into C object representations. The primary purpose of json_tokener_parse is to provide a simple interface for parsing JSON strings.

Based on the function parameters, it does the following:

- It takes a single parameter str, which is a string (const char) containing the JSON data to be parsed.
- The function calls json_tokener_parse_verbose, passing the input string and the address of the error variable.
- It returns the struct json_object received from json_tokener_parse_verbose, which represents the parsed JSON data as a C object.

Based on our understanding, this function provides a straightforward way to parse JSON strings without dealing with explicit error handling.

By fuzzing this function, we can potentially uncover issues related to parsing errors, memory corruption, buffer overflows, or other security vulnerabilities that might arise from processing malformed JSON input.

3.3 Fuzzing Harness and Setup

Just like before, we modified the fuzzing harness to target the json_tokener_parse function. It focuses on testing the json_tokener_parse function and related JSON object manipulation functions.

```
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
        const char *data1 = reinterpret_cast<const char *>(data);
        json_tokener *tok = json_tokener_new();
        json_object *obj = json_tokener_parse_ex(tok, data1, size);

        if (json_object_is_type(obj, json_type_object)) {
                json_object_object_foreach(obj, key, val) {
                        (void)json_object_get_type(val);
                        (void)json_object_get_string(val);
                }
        }
        (void)json_object_to_json_string_ext(obj, JSON_C_TO_STRING_PRETTY | JSON_C_TO_STRING_SPACED);

        json_object_put(obj);
        json_tokener_free(tok);
        return 0;
}
```

The harness is designed to:

- Test the basic parsing functionality of JSON-C
- Test string conversion and type-checking
- Ensure proper memory management by creating and freeing objects

(For a detailed view, please look at the files provided in the artifacts.)

3.4 Fuzzing Process

For our fuzzing, we outlined the steps below:

3.4.1 Inputs and Seed Files

To improve the fuzzing process, we filled the corpus folder with real-world JSON files to act as initial inputs to guide the fuzzer to explore different paths. We aimed to provide diverse and complex examples just like task 1. We downloaded these files from the following GitHub repository that provides a wide range of examples:

- JSON Corpus
    - GitHub Link: https://github.com/sdbs-uni-p/json-schema-corpus.git
    - This corpus is part of a C library designed for text corpus analysis.. It's particularly well-suited for our purpose due to its focus on JSON processing and handling of large data files. It includes patterns from various sources such as:
        - Diverse JSON structures
        - Edge cases
        - Newline-Delimited JSON

- Large File Handling

We populated the corpus folder with around **100** files to provide a wide range of cases.

Here are some examples to help us understand why this is important:

File	Description	File Content
1	Defines a schema for a "HasRequest" object	`{ "$schema": "http://json-schema.org/schema#", "title": "HasRequest", "type": "object", "properties": { "key": { "type": "string", "pattern": "^[a-zA-Z0-9_\\.]+$", "maxLength": 12 } ...`
2	Defines a schema for a "RewindResponse" object	`{ "$schema": "http://json-schema.org/schema#", "title": "RewindResponse", "type": "object", "$ref": "resource://mdstudio/db/cursor/v1" }`
3	Defines a schema for an Apple HealthKit correlation sample	`{ "$schema": "http://json-schema.org/draft-04/schema#", "description": "An Apple HealthKit correlation sample that represents "type": "object", "references": [{ "description": "The class reference for HKCorrelationSample.", "url": "https://developer.apple.com/library/ios/documentation/Heal }], "definitions": { "hk_correlation_type": { "$ref": "hk-correlation-type-1.x.json" }, ...`
4	Defines a schema for a "ReplaceOneRequest" object	`{ "$schema": "http://json-schema.org/schema#", "title": "ReplaceOneRequest", "type": "object", "properties": { "collection": { "$ref": "resource://mdstudio/db/collection/v1" },`
5	Defines a schema for an object with a "corrections" array	`{ "$schema": "http://json-schema.org/draft-04/schema#", "type": "object", "required": [ "corrections" ], "properties": { "corrections": { "type": "array", ...`

These are only some examples that we chose to show the kinds of different files that are being used.

These files provide a different range of inputs to explore so the fuzzer can mutate and find paths that may introduce bugs. Hence, providing an initial input would also speed up the process.

3.4.2 Execution (Fuzzing Configuration)

We configured AFLplusplus with specific parameters to optimize its performance and effectiveness in testing JSON-C.

Using the following command we executed AFLplusplus:

```
afl-fuzz -i inputs -o output_dir -x json.dict -L 0 -t 250+ -D -m none -- ./json_tokener_fuzz_harness @@
```

- A JSON-specific dictionary (json.dict) was used to enhance input mutation relevance.
- A timeout of 250 milliseconds per execution was set, with no memory limit imposed.

3.4.3 Analysis of Fuzzing Results

```
  american fuzzy lop ++4.00c {default} (./json_tokener_fuzz_harness) [fast]
┌─ process timing ──────────────────────────────────────┬─ overall results ─────┐
│        run time : 1 days, 14 hrs, 32 min, 56 sec      │  cycles done : 34.6k  │
│   last new find : 0 days, 2 hrs, 57 min, 14 sec       │ corpus count : 878    │
│last saved crash : none seen yet                       │saved crashes : 0      │
│ last saved hang : none seen yet                       │  saved hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage─┴───────────────────────┤
│  now processing : 852.0 (97.0%)    │    map density : 6.60% / 25.39%       │
│  runs timed out : 0 (0.00%)        │ count coverage : 6.25 bits/tuple      │
├─ stage progress ────────────────────┼─ findings in depth ───────────────────┤
│  now trying : MOpt-splice 8        │ favored items : 93 (10.59%)           │
│ stage execs : 82/1152 (7.12%)      │  new edges on : 109 (12.41%)          │
│ total execs : 1.09G                │ total crashes : 0 (0 saved)           │
│  exec speed : 8269/sec             │  total tmouts : 177 (44 saved)        │
├─ fuzzing strategy yields ───────────┴───────────────┬─ item geometry ───────┤
│   bit flips : disabled (default, enable with -D)    │    levels : 9         │
│  byte flips : disabled (default, enable with -D)    │   pending : 877       │
│ arithmetics : disabled (default, enable with -D)    │  pend fav : 93        │
│  known ints : disabled (default, enable with -D)    │ own finds : 876       │
│  dictionary : n/a                                   │  imported : 0         │
│havoc/splice : 81/119M, 236/189M                     │ stability : 97.52%    │
│py/custom/rq : unused, unused, unused, unused        ├───────────────────────┘
│    trim/eff : 17.26%/56.8k, disabled                │          [cpu000: 25%]
└─────────────────────────────────────────────────────┘          ^C
```

- Run Time: 1 day, 14 hours, 32 minutes and 56 seconds.
- Executions: 1.125 billion executions, with an average of 7,833.81 executions per second.

- Corpus Information:
  - Corpus Count: 885 total test cases in the corpus.
  - Unique Paths: AFLplusplus discovered 405 unique execution paths out of 1,591 total edges.
  - Map Coverage: Achieved a bitmap coverage of 25.46%, indicating significant exploration of potential paths.
- Crashes and Hangs:
  - Crashes Found: 0
  - Hangs Found: 0
- Additional Metrics:
  - Pending Favors: 91 favored inputs pending further exploration.
  - Stability: The fuzzer achieved 97.53% stability, showing that inputs produced consistent results across runs.

Just like task 1, AFLplusplus was not able to find any crashes or hangs. This may be due to rigorous testing that has already been done on such an open-source project.

3.5 Results and Analysis

After we ended the fuzzing, the fuzzer generated 885 files under queue.

We chose a few different files that led to interesting behaviours that will help us to understand the edge cases and how AFLplusplus is mutating different files to use them as inputs.

File	Observation	File Content

1	It has deeply nested objects and looks like it has some Unicode characters. It may challenge JSON's depth handling.	`[[[{"nu": {"u#  array": {"#  ar ay": {"\uD888\uD8888 8`
		`  \uD888\uD8888 8\uD8888 8   \uD88888 8`
		`  \uD888\uD8888 8\uD8888 8   \uD888\uD8888`
		`  array": {"#  ar ay": {"\uD888\uD8888 8`
		`  \uD888\uD8888 8\uD8888 888 U\uD888\uD8888 8`
		`  88   8  88\uD8888  array": {"#  ar ay": {"\uD888\uD8888 8`
		`  \uD888\uD8888 8\uD8888 888 U\uD888\uD8888 8   88   8  v`
		`  ...`

2	It contains a structure that has missing parts, testing the parser's ability to handle unexpected end of input.	`{"nurray":[     {"n#  arrTy": {"\uE8E89888u#  arrTy":` `{"\uE8E89888  !8`

3	It features nested arrays and objects with Unicode characters.	`[       0,[     [[[[    {"nu#ray": {"\u888888": {"rray":`
		`{"\u888888\u88888X8  gy": {"\u888888":`
		`{"\u88888888888\u88888>8y":`
		`{"\u8888888888": {"\u88888": {"\v`
		`...`

4	It presents a minimalistic, slightly malformed structure, useful for testing basic error detection.	`[[{"numer":{"nuyb  ":{": ":[[{"num er":{"nuyb  e ":{": ": {v`
5	It includes Unicode characters and a mix of valid and invalid object structures.	`{"nu¿#  a}a": {"\uD888{ nu": {"u#  ar}ay": {}}}}}`
6	It tests nested arrays with Unicode characters and truncated strings.	`[  0,0,[  0,0,{"n ": {"u# ]array": {"QYD8ay": {"\uD888\uDE88a8ay": {"\uD888\uDE88array": {"\uD888\uDE888 188\uDE88  ray": {"\uD888\uDE888 ...`
7	It contains a complex array of numbers, including negative zeros and very large numbers, testing number parsing and precision handling.	`{"nu": [[[[000600,0,-0,600,-0,000,-0, 0,23,-0,0, 0,00,-0,000,-0,000,-0,0,23 ,0,[[  0,00,-0,600,-0,000,-0,0,23 ,-0,0,  0,00,000,-0,0,23,0,[[  0,00,-0,600,-00,00, 999999999999999999999999999999999999 99999999999999999999999999999999999 999999999999999,000,-0,000,-0,0,23,0,[[ ...`
8	It focuses on string handling with repeated null characters and Unicode escapes.	`" \u00000000000000000000 0 0000000000\u00000 \u00000000000000000000 0 0000000000\u00000000000000000000 0  0 000000000b eee  0000b eee  00000000000 0  0  000000000b eee  0000b eee  eee e  v`
9	It presents a complex nested structure with many floating-point numbers in scientific notation, testing both structural parsing and number handling.	`[[[],[002,  002,0,22.+E+,0,[ 0.2,0,22.+E,  002,0,22.+E+,22.+E+,0,[ 0.2,0,22.+E+,02,2,0,22.+E+,0,[  0E+,0,[ 002,  002,0,22.+E+,22.+E+,0,[ 0.2,0,22.+E+,02, 002,0,22.+E+,0,[ 0E+,0,[ 002,  002,0,22.+E+,0,[ ...`
10	It includes metadata from AFL (execution count, operation type) followed by a simple JSON-like structure, testing the parser's ability to ignore leading garbage.	`6972313,execs_199792340,op_MOpt_core_splice,rep_8 3number": 123, "arrar": 123, "arranemer": 123, "aranumber": numberZ: 123v`

These are only some of the files we analyzed. For some other files, you may check the artifacts submitted.

Our key observations are:

- The use of Unicode characters in the generated files suggests that JSON-C's Unicode handling is being thoroughly tested.
- Several files with deeply nested structures may help identify any limits in JSON-C's ability to handle complex hierarchies.

- The variety of number formats, including edge cases like -0 and very large numbers, tests the robustness of JSON-C's number parsing routines.
- Malformed structures in the generated files test JSON-C's ability to detect and recover from parsing errors.
- Long strings and repeated characters were used to check issues related to buffer management or overflow.

3.6 Challenges and Reflection

Initial Challenges

1. Setup and Configuration: Ensuring that JSON-C was correctly instrumented for AFLplusplus required verifying that compilation flags were configured to support AFL's instrumentation, such as -fsanitize options when applicable.
2. Curating Input Corpus: The initial input corpus was curated using sample JSON files, allowing AFLplusplus to start with structured inputs that resemble typical JSON data. This initial set provided a strong basis for mutation and deeper coverage.
3. Handling Malformed Inputs: JSON-C, designed for well-formed JSON inputs, occasionally faced issues with AFLplusplus's highly mutated inputs, which included malformed JSON. This behaviour was managed by monitoring error handling within the library to ensure robust parsing without crashes.

Performance and Resource Management

1. Execution Speed: AFLplusplus maintained an average execution speed of 7,833.81 executions per second, which was sufficient for covering a substantial number of paths within a reasonable timeframe.
2. Memory and CPU Usage: Configured with unrestricted memory (-m none), AFLplusplus used available resources efficiently without significant slowdown or excessive memory consumption.

Fuzzing Strategy

1. Mutation Strategies: AFLplusplus employed various strategies that effectively explored the JSON-C library's handling of diverse inputs. The mutation approach allowed for an extensive exploration of JSON parsing edge cases.
2. No Bugs Found: Despite comprehensive testing, no crashes or hangs were observed during the fuzzing process. This outcome may reflect the robustness of JSON-C's JSON parsing capabilities or indicate a lack of deeper vulnerabilities within the reachable code paths during testing.

3.7 Reflection on Fuzzing Process and Tools

1. Strengths of AFLplusplus:
    - Coverage-Guided Approach: AFLplusplus's path exploration and edge discovery mechanisms allowed effective exploration of JSON-C's input processing, identifying unique paths and validating robustness.
    - Configurability: AFLplusplus offered extensive customization options for timeout, memory, and mutation strategies, which allowed a tailored fuzzing approach suited to JSON-C's structure.

2. Weaknesses and Limitations:
    - No Crash Findings: The lack of crashes or hangs suggests that AFLplusplus either did not reach vulnerable code or JSON-C's error handling mechanisms sufficiently managed erroneous inputs.
    - Limited Path Coverage: Despite AFLplusplus's efforts, bitmap coverage was capped at 25.46%. This limitation might be due to AFL's inability to reach certain code paths within JSON-C, potentially due to conditional checks.

4. Comparison Between LibFuzzer and AFLplusplus

Based on the two projects we analyzed with these different tools, we will provide a comparison between LibFuzzer (used on http-parser) vs AFLplusplus (used on JSON-C), highlighting their strengths, weaknesses, and overall effectiveness in our fuzzing process:

4.1. Challenges Faced as a Group:

- Setting up both tools was complicated. For LibFuzzer, enabling sanitizers like AddressSanitizer required compiling with specific flags and careful harness setup to target `http_parser_execute`. In AFLplusplus, configuring dictionary files and ensuring JSON-C's compilation with instrumentation was challenging. Time constraints added pressure, as we needed to address unexpected configuration errors on multiple occasions.
- Populating an effective initial corpus for each tool demanded substantial research. In particular, LibFuzzer's performance heavily depended on high-quality inputs, which we got from the FuzzDB and HTTPcorpus repositories. For AFLplusplus, we focused on JSON structural variety, using inputs that could cover edge cases in JSON parsing.
- Both tools required prolonged runtime (48 hours for LibFuzzer and 40 hours for AFLplusplus), necessitating careful monitoring of system resources. LibFuzzer and

AFLplusplus's output corpus size became a challenge as it generated thousands of files (885 unique test cases) that consumed considerable disk space.

4.2. Effort Required for Each Tool:

- LibFuzzer: High initial setup effort due to the need for compiling with sanitizers and configuring the fuzzing harness. Its reliance on input quality also required considerable time to curate effective inputs. Execution time was substantial, with over 22 billion executions in 48 hours.
- AFLplusplus: Setup was relatively quicker since AFLplusplus managed corpus generation automatically, but analyzing the results required significant effort due to the high volume of outputs and lower path coverage compared to LibFuzzer.

4.3. Strengths and Weaknesses:

LibFuzzer:

- Strengths: Its in-process fuzzing capability and integration with AddressSanitizer made it effective for memory corruption and buffer overflow detection. With 40,000+ unique inputs generated and 66.6% line coverage, LibFuzzer excelled in efficient path exploration for single functions like the one we tested `http_parser_execute`.
- Weaknesses: Limited to function-level fuzzing; full-program testing is challenging. Its dependency on initial inputs made it prone to getting "stuck" in paths with limited diversity.

AFLplusplus:

- Strengths: Coverage-guided and highly configurable, AFLplusplus's performance was notable with JSON-C, producing 885 unique test cases and reaching 25.46% bitmap coverage. Its robust mutation strategies allowed it to effectively explore a diverse range of JSON structures.
- Weaknesses: Lower path coverage (25.46%) indicates a possible bottleneck in reaching deeper code paths. The lack of crashes in JSON-C suggests AFLplusplus might not have fully stressed the library's limits.

4.4. Reflected in Results:

LibFuzzer's fast execution rate (127,446 executions per second) and high corpus growth showed its efficiency in generating diverse inputs. However, the reliance on quality initial inputs meant that it changed after achieving initial path exploration, limiting further unique discoveries.

AFLplusplus demonstrated a high stability rate (97.53%) and was capable of producing more structured JSON inputs, but its path coverage (25.46%) suggests limitations in exploring nested

or conditional paths within JSON-C. Its mutation-based strategy was effective but did not reveal any vulnerabilities or edge cases that could lead to crashes.

4.5. Crashes and Hangs:

Outcome: Neither tool produced crashes or hangs during the extended runtime. Given that http-parser and JSON-C are widely used and mature libraries, it's likely they have already undergone extensive testing.

Analysis: The lack of crashes suggests either a high level of robustness or that the fuzzers were unable to execute specific, vulnerable code paths. JSON-C's stability across 1.1 billion executions with AFLplusplus indicates resilient error handling against malformed JSON inputs.

4.6. Reasons for Fewer Bugs:

Mature Codebases: Both libraries have mature, well-tested codebases with significant contributions and security focus. For example, JSON-C has over 10,600 lines of code and extensive prior testing, making new bug discovery challenging.

Existing Fuzzers: Both projects have existing fuzzing tools, which likely cover the most vulnerable paths. This overlap with established testing methods likely reduced the discovery of new issues.

4.7. Did the Fuzz Tester Get Stuck on Irrelevant Code?

LibFuzzer occasionally became repetitive, especially on edge-case HTTP paths, suggesting that it reached a point with limited input structures.

AFLplusplus explored JSON constructs effectively but frequently returned to familiar patterns without delving into deeper, more conditional paths.

4.8. Impact of Tool on Testing Results:

LibFuzzer's strength in handling function-specific fuzzing yielded good results for http-parser's `http_parser_execute`. However, its performance depended on dictionary and corpus quality.

AFLplusplus excelled in generating a broader range of inputs and identifying interesting JSON edge cases, but its lower path coverage may reflect limitations in JSON-C's structure or AFLplusplus's mutation strategy.

4.9. Evidence to Contrast LibFuzzer and AFLplusplus:

Execution Speed: LibFuzzer averaged 127,446 executions per second, much faster than AFLplusplus's 7,833 executions per second, reflecting LibFuzzer's efficiency in high-throughput, function-level testing.

- Path Coverage: LibFuzzer achieved higher line coverage (66.6%) on `http_parser_execute`, whereas AFLplusplus achieved only 25.46% bitmap coverage, suggesting AFLplusplus's path exploration is better suited for broader but less in-depth exploration.
- Stability: AFLplusplus exhibited higher input stability (97.53%), possibly due to JSON-C's resilience against malformed inputs, but LibFuzzer's sanitization coverage was invaluable for detecting memory issues.

5. Overall Analysis

The project illustrated the strengths and limitations of using fuzzing tools like LibFuzzer and AFLplusplus on mature, open-source projects. While both tools were effective in generating diverse test cases, they did not uncover any critical vulnerabilities, likely due to the robustness of the selected libraries, both of which had undergone prior fuzz testing.

Key observations include:

- Both tools' effectiveness was significantly influenced by the initial corpus. In our case, diverse inputs led to 40,000 unique mutations in LibFuzzer and 885 unique paths in AFLplusplus.
- Despite extensive fuzzing (22 billion executions for LibFuzzer and 1.1 billion for AFLplusplus), we observed limited path coverage, highlighting the need for further optimized inputs or more specialized fuzzing techniques.
- AFLplusplus's extensive output management and LibFuzzer's sanitization demands highlighted the importance of resource optimization in long-running fuzzing sessions.

The absence of crashes across both projects suggests that high-quality initial seeds combined with structured input dictionaries are essential. However, even with these inputs, the tools had limitations in covering all paths, especially in well-tested codebases.

6. Individual Analysis

To provide a general overview, we will answer the following questions from the exercise individually.

6.1 Dhruv

1. What specific challenges did you face personally while using the fuzz testing tools?
    - While the general challenges are mentioned as a group, I personally struggled with optimizing the fuzzing process for maximum code coverage. As seen, we provided a coverage analysis that does show the percentage of code covered. Our goal was to aim higher.
    - Interpreting the large amount of data generated during the day long fuzzing run was particularly challenging.
2. How did you approach and overcome these challenges?
    - To improve code coverage, I experimented with different seed corpus compositions, referencing the techniques mentioned in section 2.4.1 and 3.4.1
3. Were there any errors indicated by the analyses? If so, what were they?
    - While no crashes were found, I observed several instances where the parser behaved unexpectedly with certain malformed inputs, particularly those combining multiple HTTP versions where it took slightly longer.
4. How did you analyze and interpret the errors or issues you encountered?
    - I utilized the coverage analysis to identify which code paths were triggered by unexpected inputs.
5. If fewer than *k* bugs were found, what is your personal take on why this happened?
    - Building on the reasons we've mentioned as a group, I think our fuzzing strategy, while extensive, may have not sufficiently triggered real-world attack cases that could potentially uncover more subtle vulnerabilities.

6.2 Vraj

1. What specific challenges did you face personally while using the fuzz testing tools?
    - In addition to the setup challenges, I found it difficult to balance between exploring new code paths and focusing on known interesting areas of the code.
    - Determining when to stop the fuzzing process and how to prioritize the analysis of the generated inputs was a personal challenge. For LibFuzzers, I did different runs of 4 hours, 8 hours and a final run of 48 hours to ensure the generated inputs will be useful for our analysis. For the first run which was around 10 minutes, I noticed no inputs were being generated. Hence, I had to modify the harness to ensure it provides the right inputs for the longer run.

2. How did you approach and overcome these challenges?
    - I decided to add inputs that have been widely used in such fuzzing tasks from various GitHub repositories. This approach simplified the process and allowed me to add as many files as I could. Manually writing out files was not the right approach since I lack the right knowledge required for such tools that is relatively complex.
    - I planned out an outlined to analyze the generated inputs based on their uniqueness and the code paths that were used. I wanted to ensure a wide range of examples were used. Hence, I had to go through several files before choosing the right ones to analyze in this report.
3. Were there any errors indicated by the analyses? If so, what were they?
    - While no crashes were detected, I noticed patterns in the corpus that did suggest some potential edge cases that we analyzed in sections 2.4.6 and 3.5.
4. How did you analyze and interpret the errors or issues you encountered?
    - Based on the inputs, I listed all of them done and divided it in categories so I could research on the different HTTP elements that the fuzzing was targeting. As I lacked the right knowledge of the HTTP elements, I had to read a few references.
5. If fewer than *k* bugs were found, what is your personal take on why this happened?
    - Expanding on the reflections we mentioned before, the robustness of both projects made it relatively harder for the different types of inputs that were generated by the fuzzer. Since open-source projects are developed over many years with regular updates, it would be hard to find significant bugs.

7. Conclusion

This exercise demonstrated that both LibFuzzer and AFLplusplus are valuable tools for exploring code paths and identifying potential vulnerabilities in open-source projects. While neither tool discovered new crashes or hangs, the testing experience reinforced the following insights:

- Tool Strengths: LibFuzzer is well-suited for focused function-level testing with high execution rates, especially when integrated with sanitizers, while AFLplusplus excels in mutating structured inputs and is more configurable for complex fuzzing scenarios.
- Tool Limitations: Both tools struggled to reach certain code paths in highly structured or conditional logic, likely due to limitations in mutation strategies and dependence on the corpus.

- Robustness of Selected Projects: http-parser and JSON-C have resilient codebases, possibly due to prior fuzzing efforts. This lack of new findings underscores the difficulty of discovering vulnerabilities in mature projects.

In conclusion, this fuzz testing project provided valuable hands-on experience in setting up, executing, and analyzing fuzz testing campaigns with LibFuzzer and AFLplusplus. Both tools, while effective, benefit from curated inputs and structured testing environments. As the complexity of tested software grows, fuzzing strategies will need to incorporate more advanced techniques, including hybrid fuzzing or differential testing, to improve coverage and identify vulnerabilities in already well-tested software.

(References have been provided as hyperlinks throughout the report.)